

# MISSIONARIES AND CANNIBALS PROBLEM

## Aim

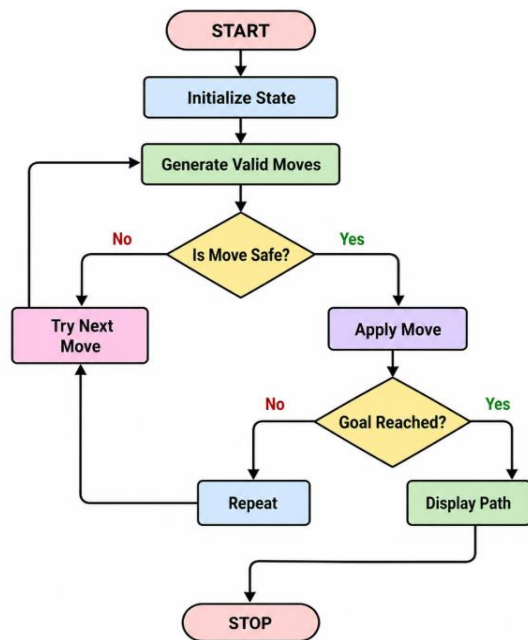
To solve the Missionaries and Cannibals problem using Artificial Intelligence search techniques.

## Objectives

- To understand state-space search.
- To transport all missionaries and cannibals safely.
- To ensure missionaries are never outnumbered.
- To find a valid sequence of moves.

## Algorithm

1. Start.
2. Initialize the starting state (3,3,1).
3. Generate all possible valid moves.
4. Check whether the move is safe.
5. If safe, move missionaries and/or cannibals.
6. Repeat until the goal state (0,0,0) is reached.
7. Display the solution path.
8. Stop.



## PROGRAM

```

from collections import deque

def missionaries_cannibals():
    start = (3, 3, 1)
    goal = (0, 0, 0)

    queue = deque([(start, [])])
    visited = set()

    moves = [(1,0), (2,0), (0,1), (0,2), (1,1)]

    while queue:
        state, path = queue.popleft()

        if state == goal:
            return path + [state]

        if state in visited:
            continue

        visited.add(state)

        m, c, boat = state

        for dm, dc in moves:
            if boat:
                new = (m-dm, c-dc, 0)
            else:
                new = (m+dm, c+dc, 1)

            nm, nc, nb = new

            if (0 <= nm <= 3 and 0 <= nc <= 3 and
                (nm == 0 or nm >= nc) and
                (3-nm == 0 or 3-nm >= 3-nc)):

                queue.append((new, path + [state]))

solution = missionaries_cannibals()
print("Solution Path:")

```

## OUTPUT

```
= RESTART: C:/Users/ATHIN/AppData/Local/Programs/Python/Python39-64/Python.exe
Solution Path:
(3, 3, 1)
(3, 1, 0)
(3, 2, 1)
(3, 0, 0)
(3, 1, 1)
(1, 1, 0)
(2, 2, 1)
(0, 2, 0)
(0, 3, 1)
(0, 1, 0)
(1, 1, 1)
(0, 0, 0)
```

## Result

The Missionaries and Cannibals problem was successfully solved using the Breadth First Search (BFS) algorithm, and all missionaries and cannibals were transported safely to the opposite bank.